

Polynomial Regression – Assignment 1

Name: Rohith Sai Manne
Roll No: BT2024144

1 Introduction and Data

In this assignment I built polynomial regression models for two problems. In Problem 1 (var1) I predicted the Net Power Score of a steam turbine from six inputs x_1 – x_6 . In Problem 2 (var2) I predicted a Thermal Anomaly Score from three coordinates x_1, x_2, x_3 . Both problems have 1000 training rows and 1000 test rows with no missing values, and every input lies in $[-1, 1]$.

In Problem 1 the target has mean 0.85, standard deviation 3.08 and range $[-8.8, 11.5]$. The linear correlations with y are all weak (x_5 : -0.22, x_6 : -0.20, x_3 : -0.13), and a straight-line fit only reaches a CV R^2 of 0.08, so the relationship is clearly non-linear. Many input values sit exactly at the limit ± 1 : 29.9–33.3% of the training values of each feature, but 47.3–52.9% of the test values (std 0.81 in test vs 0.71 in train). So the var1 test set has many more rows near the corners of the input box than the training set (Fig. 1). I kept this in mind when choosing the model.

In Problem 2 the target has mean 2.24, standard deviation 6.78 and range $[-29.8, 38.5]$. Here train and test look alike (23.7–26.4% of the values at ± 1 in train, 23.8–26.6% in test), so there is no such shift.

What I did about this. For Problem 2 nothing special was needed, so ordinary 5-fold cross-validation on the training data is a fair guide to the test score. For Problem 1 the shift matters, because a polynomial is least reliable near the edges of the input box, and 64.7% of the test rows have three or more features at ± 1 compared with 28.3% of the training rows. So for Problem 1 I did three things. First, the test inputs were never used for fitting; I only used them to measure this shift. Second, I used a penalty (Lasso or Ridge) instead of an unpenalised fit, because high-degree terms grow very fast near the edges. Third, I added an extrapolation split (Section 2) as an extra validation check that imitates the test situation, and used it to confirm the degree and the model (Section 3); higher degrees did much worse on the extrapolation split. I also looked at how the error changes with the number of features at ± 1 (Section 3). This is why I expect the Problem 1 test error to be somewhat higher than its CV error.

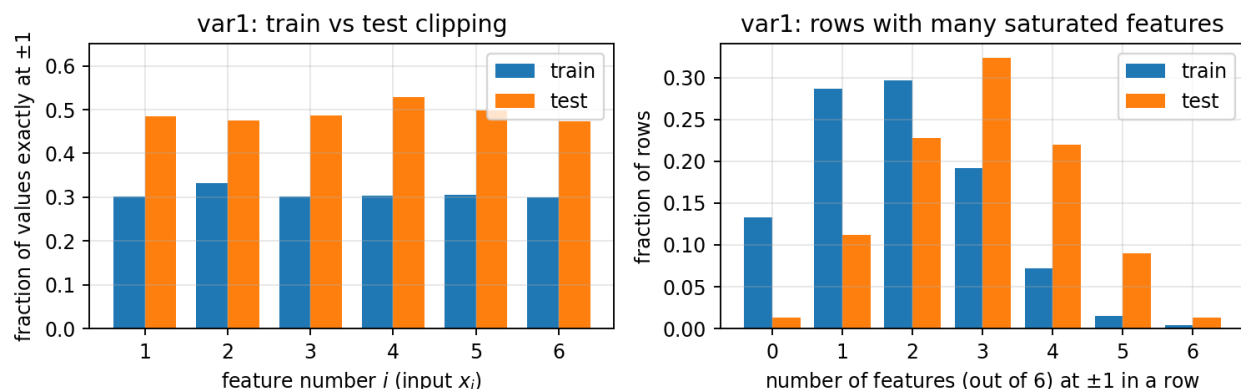


Figure 1: Problem 1, share of values at ± 1 in train and test. **Left:** the x-axis is the feature number (x_1 to x_6) and the y-axis is the fraction of that feature’s values that are exactly ± 1 ; blue bars are train and orange bars are test. Every feature is at ± 1 for 29.9–33.3% of the training rows (0.299–0.333) but for 47.3–52.9% of the test rows. **Right:** the x-axis is how many of the six features of a row sit at ± 1 (0 to 6) and the y-axis is the fraction of rows with that count. Training rows mostly have 1 or 2 such features (average 1.8), while test rows mostly have 3 or 4 (average 2.9), so the test set has many more corner-like rows.

2 Methodology

Pipeline. Every model has the same three steps: `PolynomialFeatures(d)` $\rightarrow$ `StandardScaler` $\rightarrow$ Ridge or Lasso. `PolynomialFeatures` builds all terms whose powers add up to at most d , which is the definition of degree used in the assignment. For 6 inputs and $d=5$ this gives 461 terms, and for 3 inputs and $d=10$ it gives 285. `StandardScaler` (from

scikit-learn) scales all polynomial features to zero mean and unit variance, so Ridge/Lasso penalise every term fairly (otherwise terms like x^{10} would have much smaller values than x and get unfairly penalised). The scaler is fitted on the training part of each fold only. This pipeline is built inside the `make_model` function in `src/common.py`.

Regularisation. With so many correlated terms, plain least squares overfits quickly, so I used Ridge (L2) and Lasso (L1). Ridge shrinks all coefficients, and Lasso sets many of them to exactly zero.

Choosing degree, model and α . I searched the degree (var1: 1–8, var2: 2–20), the model (Ridge, Lasso) and α together, and scored every combination with 5-fold cross-validation repeated twice. The setting with the lowest CV MSE was chosen and then refitted on all 1000 training rows to predict the test set. Lasso was only run up to $d=7$ (var1) and $d=12$ (var2) because it gets slow with many terms, while Ridge covers the full range. As an extra check I ran the whole selection again on a random 80% of the training data and scored the winner on the other 20% (hold-out).

For var1 I only looked at the test inputs to understand the shift mentioned above, using an extra validation check (Section 3): an extrapolation split, where I trained on the 70% of rows closest to the origin and validated on the 30% farthest away.

3 Results: Problem 1 (Net Power Score)

CV MSE drops quickly until degree 3–4 and is lowest at degree 5 (Table 1, Fig. 2); after that it goes up again, so lower degrees underfit and higher ones overfit. Lasso is clearly better than Ridge from degree 4 on (0.35 vs 0.54 at $d=5$). Of the 461 terms Lasso keeps 111, and the biggest coefficients belong to x_1x_5 , $x_2^3x_3^3$, x_6^2 and x_3x_6 , so the output depends on a limited set of interactions.

degree	1	2	3	4	5	6	7	8
Ridge (best α)	8.71	2.97	0.98	0.77	0.54	0.62	0.67	0.76
Lasso (best α)	8.71	2.96	0.96	0.65	0.35	0.37	0.39	-

Table 1: Problem 1, best CV MSE for each degree.

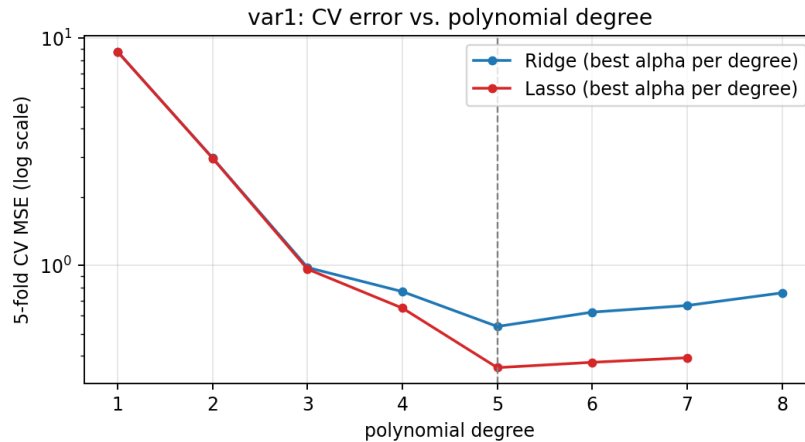


Figure 2: Problem 1, CV MSE against polynomial degree (log scale).

Chosen model: Lasso, $d=5$, $\alpha=0.01$. CV MSE = 0.354 ± 0.008 , CV $R^2 = 0.962$, training MSE = 0.265, hold-out MSE = 0.312 ($R^2 = 0.956$). Test predictions range from -10.7 to 14.2.

Effect of the train/test shift.

- The out-of-fold error is larger for rows with many values at ± 1 : MSE 0.35 for rows with at most one such value, 0.44 for rows with three or more. So I expect the real test MSE to be a bit higher than the CV MSE.
- In the extrapolation split Lasso $d=5$ was again best (MSE 0.78–0.84), while higher-degree models got much worse (Ridge $d=7$ is above 5). This is the main reason I did not go to a higher degree.

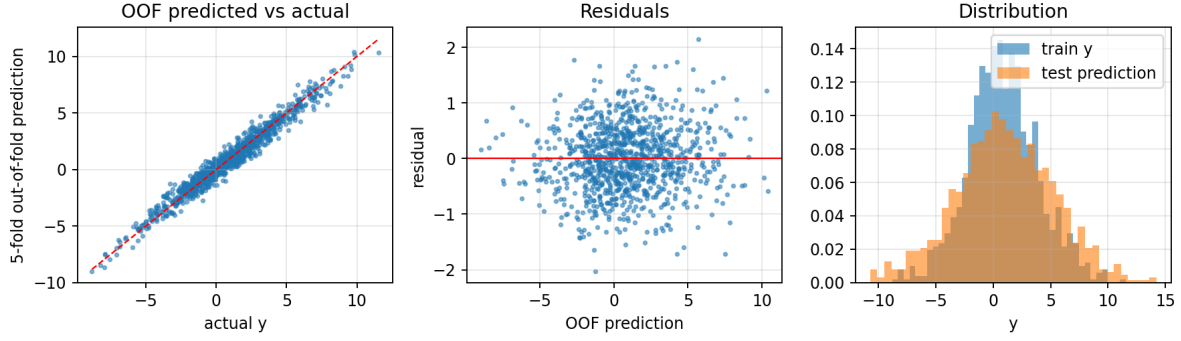


Figure 3: Problem 1, out-of-fold predicted vs actual (left), residuals (middle), and test predictions vs training y (right).

4 Results: Problem 2 (Thermal Anomaly Score)

The assignment says this surface can need a high degree, and the data agree: CV MSE falls from 21.9 at $d=2$ to 0.25 at $d=8$ and then stays flat at about 0.24 for $d=9-12$ (Table 2, Fig. 4). The lowest value is at degree 10 with Ridge $\alpha=1$. Degrees 10, 11 and 12 are within 0.002 of each other, so I took the smallest one. After $d=12$ it slowly gets worse (0.276 at $d=20$, where there are 1770 terms for only 1000 rows). Here Ridge beats Lasso (0.239 vs 0.270), which fits a smooth function where many terms matter, not a sparse one.

degree	2	4	6	7	8	9	10	11	12	14	16	18	20
Ridge	21.87	3.62	0.52	0.31	0.25	0.24	0.24	0.24	0.24	0.25	0.25	0.26	0.28
Lasso	21.87	3.62	0.61	0.42	0.32	0.29	0.27	0.27	0.27	-	-	-	-

Table 2: Problem 2, best CV MSE for each degree.

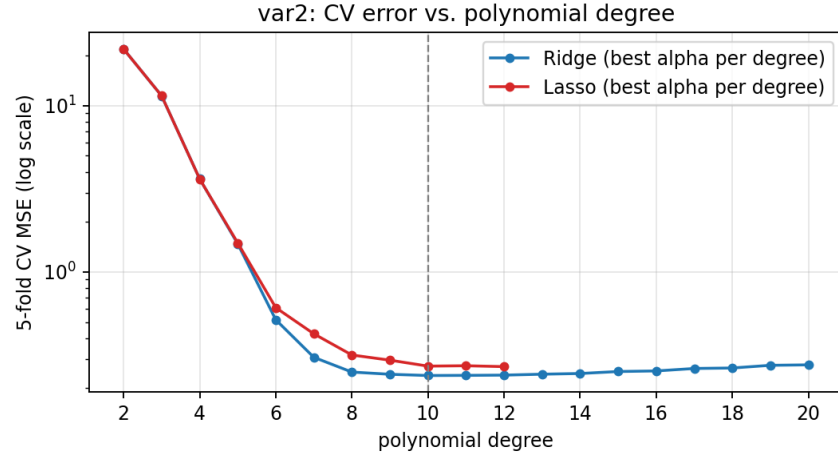


Figure 4: Problem 2, CV MSE against polynomial degree (log scale).

Chosen model: Ridge, $d=10$, $\alpha=1$. CV MSE = 0.239 ± 0.009 , CV $R^2 = 0.9947$, training MSE = 0.164, hold-out MSE = 0.246 ($R^2 = 0.9939$). Test predictions range from -27.0 to 38.6, and the actual-vs-predicted plot (Fig. 5) lies on the diagonal with no visible bias in the residuals.

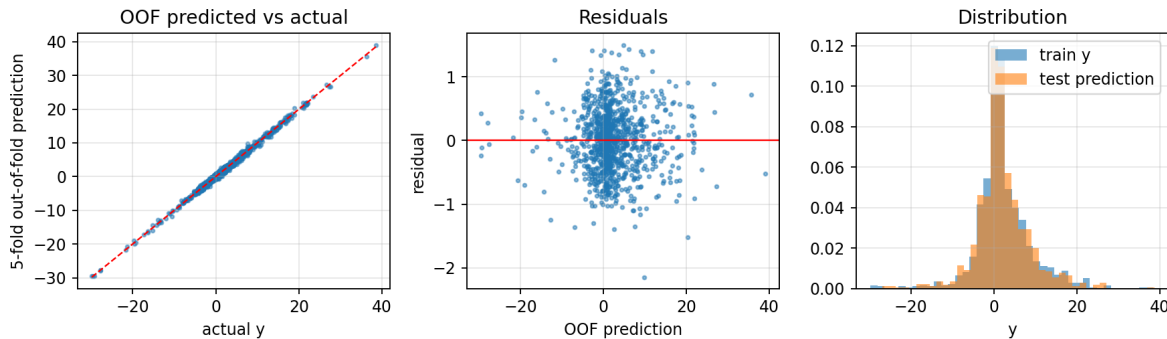


Figure 5: Problem 2, out-of-fold predicted vs actual (left), residuals (middle), and test predictions vs training y (right).

5 Discussion

Bias–variance. In both problems the CV error first falls as the degree grows (the model is too simple to follow the data) and then rises again once there are too many terms for 1000 points. Problem 1 turns around early ($d=5$) because it has six inputs, so the number of terms grows fast; Problem 2 has only three inputs and can use a much higher degree ($d=10$) before it overfits.

Why regularisation matters. Plain least squares works for a while and then breaks. In Problem 1 its CV MSE is 0.98 ($d=3$), 0.81 ($d=4$), 1.88 ($d=5$) and 70 ($d=6$). In Problem 2 it is 0.26 ($d=8$), 0.35 ($d=10$), 1.1 ($d=12$) and 5.2 ($d=13$), while Ridge stays flat over the same range.

Ridge vs Lasso. The two problems preferred different penalties. Problem 1 worked best with Lasso, which removed most terms, and Problem 2 worked best with Ridge, which keeps all terms but makes them smaller. I also tried ElasticNet ($\text{var1: } 0.374, \text{var2: } 0.263$), which was slightly worse than the chosen models, so I stayed with the simple pipeline.

How good is it? For Problem 2 the training MSE (0.164) and CV MSE (0.242) are close, the residuals look random and the error is about the same whether 0, 1 or 2 inputs are at ± 1 (0.21–0.27), and even a highly flexible Gaussian-process model could not do better (CV MSE 0.32), proving that our polynomial model successfully captured all complex patterns in the data. There are five groups of repeated input rows in the training file (13 rows) which suggest a noise level of roughly 0.05, but that is too few rows to trust, so a little room may be left. For Problem 1 the CV numbers may be a bit optimistic because of the shift towards corner points, as shown in Section 3.

Limitations. The test labels are hidden, so all scores above are estimates. Neighbouring settings (e.g. $d=10$ vs $d=11$ in Problem 2) are within CV noise, so I picked the lowest CV MSE and, when tied, the smaller degree. I only searched Lasso up to $d=7$ (var1) and $d=12$ (var2); Ridge showed no gain beyond that range, so I do not expect to have missed a better setting.

6 Conclusion

problem hold-out MSE / R^2	degree	model	α	terms (non-zero)	CV MSE	CV R^2
1 (Net Power Score) 0.312 / 0.956	5	Lasso	0.01	461 (111)	0.354 ± 0.008	0.962
2 (Thermal Anomaly) 0.246 / 0.994	10	Ridge	1.0	285 (285)	0.239 ± 0.009	0.995

Table 3: Chosen models (CV = 5-fold cross-validation repeated twice, $\pm$ is the standard error over folds).

For Problem 1 I chose a degree 5 Lasso model and for Problem 2 a degree 10 Ridge model, both selected by cross-validation using the training data only. They reach CV R^2 of 0.962 and 0.995. For Problem 2 I expect the test score to be close to the CV score, and for Problem 1 it may be somewhat worse because of the extra corner points in the test inputs.

7 Folder Structure

The repository is organised as follows:

- **code/src/**: Contains all the Python scripts.
 - **common.py**: Shared utilities for data loading, model pipelines, and cross-validation.
 - **eda.py**: Exploratory data analysis (generates the train/test shift figure).
 - **select_model.py**: Performs the degree, model, and α grid search via 5-fold CV.
 - **train_predict.py**: Fits the chosen model on all training data and writes final predictions.
 - **gp_check.py**: (Reference) Gaussian-process baseline comparison.
- **code/data/**: Contains the provided training and test CSV files.
- **code/predictions/**: Stores the final generated prediction CSVs.
- **code/results/**: Stores the CV grids, chosen configurations (JSON), and validation metrics.
- **code/figures/**: Stores the diagnostic and evaluation plots used in this report.

8 Commands to Implement

GitHub Repository Link: https://github.com/Rohithmanne13/ML_Assignment_Polynomial_Regression

Run these from inside the `code` folder, in this order:

```
pip install -r requirements.txt (install dependencies)
python3 src/eda.py (train/test shift check + generates figures)
python3 src/select_model.py -var 1 (degree / model /  $\alpha$  search for Problem 1)
python3 src/train_predict.py -var 1 (fit the chosen model and write predictions for Problem 1)
python3 src/select_model.py -var 2 (degree / model /  $\alpha$  search for Problem 2)
python3 src/train_predict.py -var 2 (fit the chosen model and write predictions for Problem 2)
```

Reference check used in the report: `python3 src/gp_check.py` (Gaussian-process baseline comparison).

The prediction files are `predictions/BT2024144_pred_var1.csv` and `predictions/BT2024144_pred_var2.csv`.